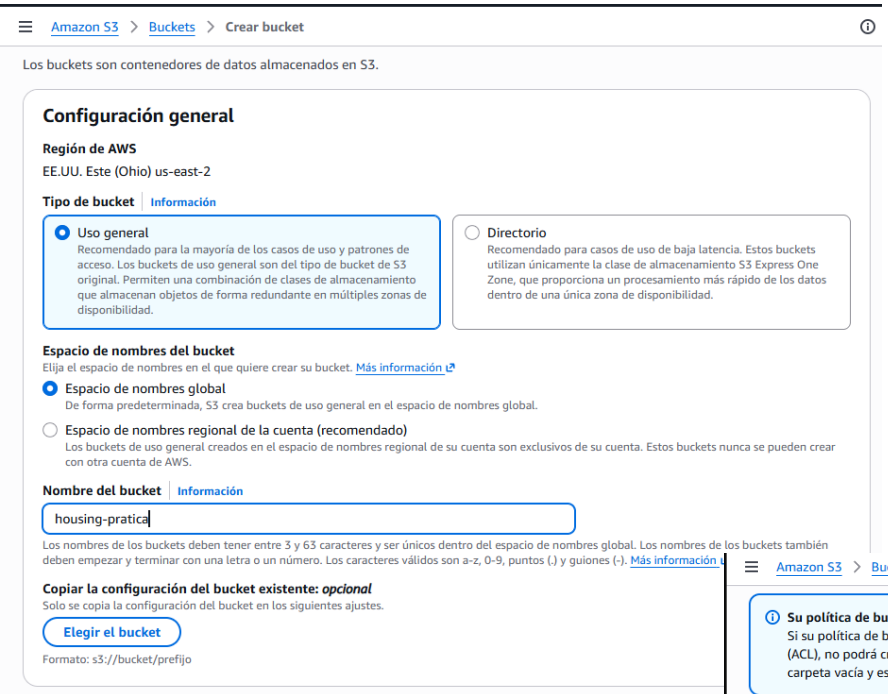
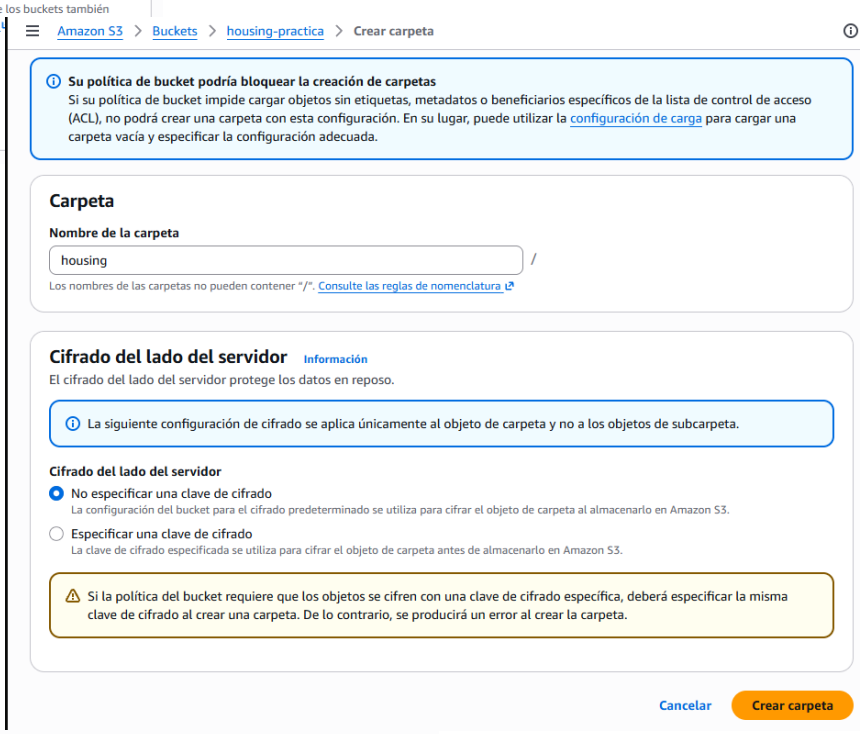
ARADNA NÚÑEZ TORRES 16 de octubre de 2026

ESTRUCTURAS Y CONCEPTOS BÁSICOS DEL DATA LAKE

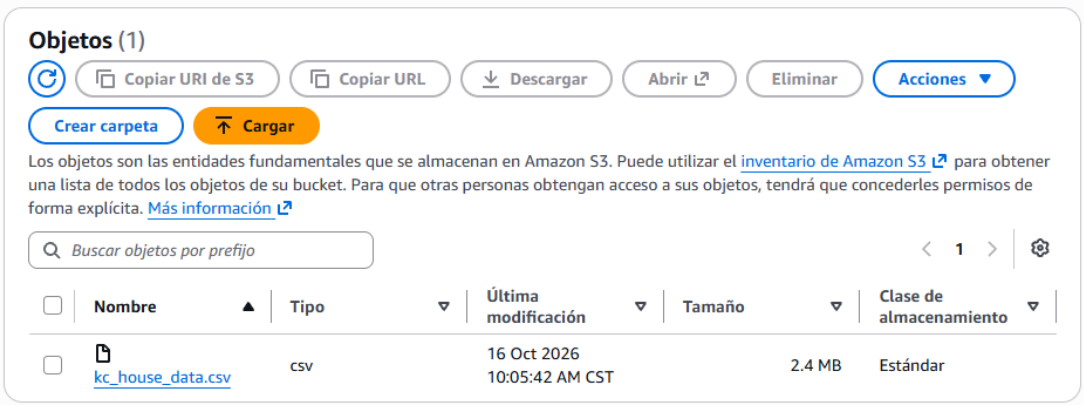
En esta práctica se pondrá de simulación un negocio de bienes raíces teniendo a un cliente que solicita información sobre una casa. Igualmente se realizarán conexiones desde AWS poniendo en práctica lo aprendido de como se compone un Data Lake, así como la exploración de la información del mismo archivo en el entorno de Python



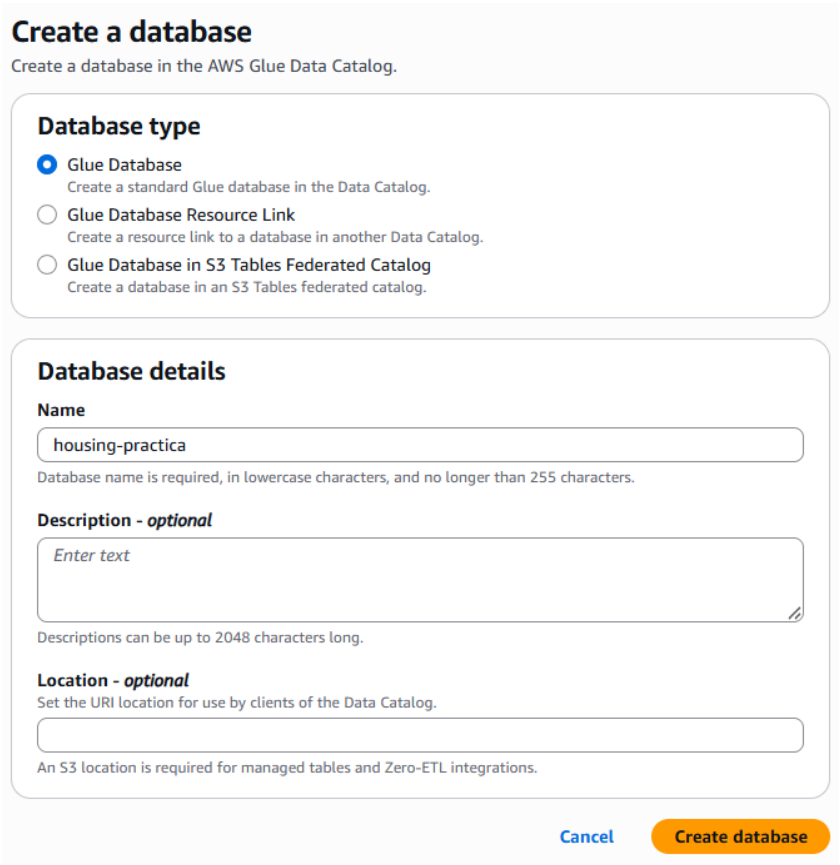
Para hacer el data lake lo que se tiene que realizar principalmente es un bucket que tendrá el nombre de la practica y en su interior tendrá la una carpeta que dirigirá hacia el archivo que se utilizará más adelante



Aquí es donde se realizará la subcarpeta que permitirá el almacenamiento del nuevo archivo

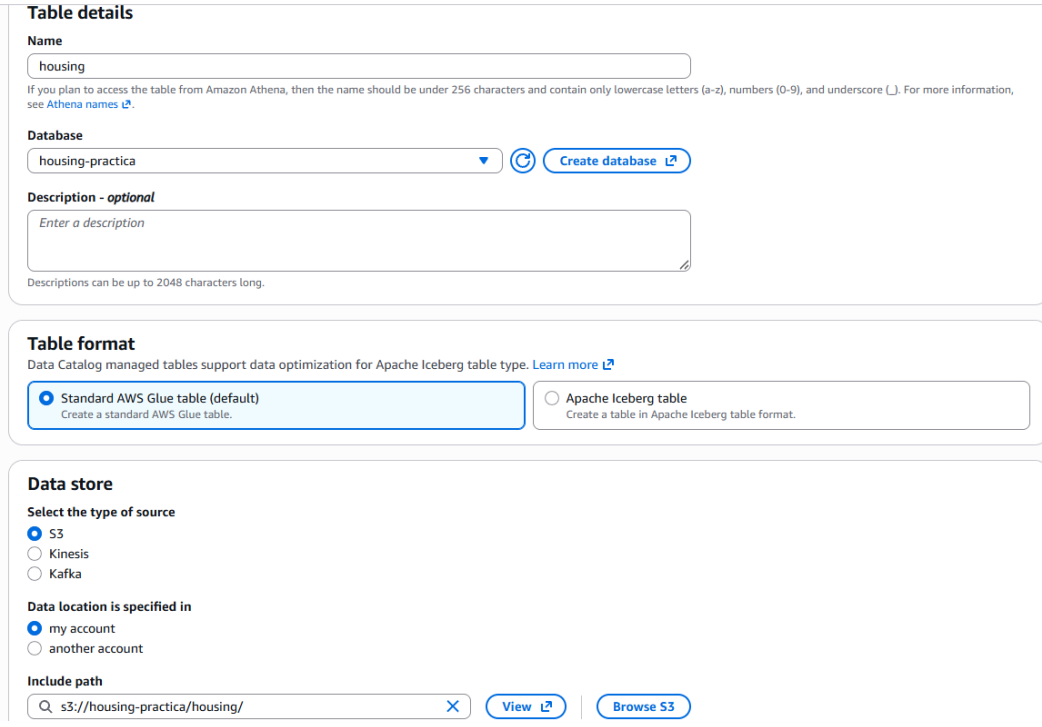


Se sube el archivo y ahora quedaría como objeto dentro de la carpeta que se realizó



Ahora en Glue se crea la base de datos donde incluirá el esquema de la tabla cargada en S3

Ahora se realiza la configuración de la tabla, señalando tipos de formato, fuentes y el directorio de la información que se ingresara a la tabla en pasos próximos



ARADNA NÚÑEZ TORRES

16 de octubre de 2026

Schema (21)

Edit schema as JSON | Edit schema

View and manage the table schema.

Filter schemas

1 2

#	Column...	Data type	Partitio...	Comment
1	id	bigint	-	-
2	date	string	-	-
3	price	float	-	-
4	bedrooms	tinyint	-	-
5	bathrooms	float	-	-
6	sqft_living	smallint	-	-
7	sqft_lot	int	-	-
8	floors	float	-	-
9	waterfront	tinyint	-	-
10	view	tinyint	-	-
11	condition	tinyint	-	-
12	grade	tinyint	-	-
13	sqft_above	smallint	-	-
14	sqft_basem...	smallint	-	-
15	yr_built	smallint	-	-
16	yr_renovated	smallint	-	-
17	zipcode	int	-	-
18	lat	float	-	-
19	long	float	-	-

Se realiza el esquema (es muy necesario si no se realiza marca error al momento de visualizarlo en Athena) con su respectivo dato, que este fue obtenido en apoyo de MSSQL

Modificar columnas

Esta operación generó el siguiente esquema de tabla. Compruebe si el esquema es preciso y, si no lo es, haga las oportunas modificaciones.

Nombre de columna	Tipo de datos	Clave principal	Permitir valores NULL
id	bigint	☐	☐
date	nvarchar(50)	☐	☐
price	float	☐	☐
bedrooms	tinyint	☐	☐
bathrooms	float	☐	☐
sqft_living	smallint	☐	☐
sqft_lot	int	☐	☐
floors	float	☐	☐
waterfront	bit	☐	☐
view	tinyint	☐	☐
condition	tinyint	☐	☐
grade	tinyint	☐	☐
sqft_above	smallint	☐	☐
sqft_basement	smallint	☐	☐
yr_built	smallint	☐	☐
yr_renovated	smallint	☐	☐
zipcode	int	☐	☐
lat	float	☐	☐
long	float	☐	☐

Tables (1)

View and manage all available tables.

Filter tables

Name	Database	Location	Classific...
housing	housing-practica	s3://housing-practica/housing/	CSV

De esta forma es como tiene que mostrarse (de igual manera al seleccionar la localización, no de se debe seleccionar)

Set crawler properties

Crawler details Q help

Name

prac-housing-crawler

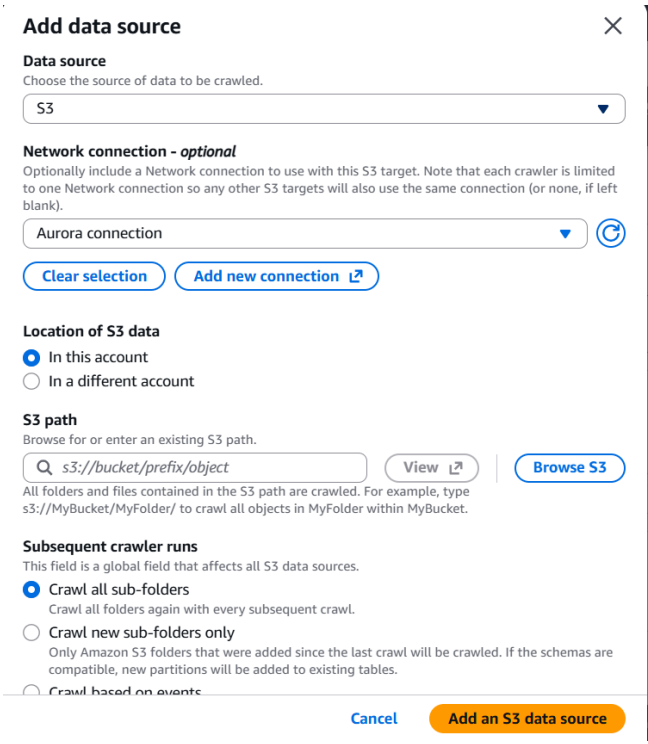
Name can be up to 255 characters long. Some character set including control characters are prohibited.

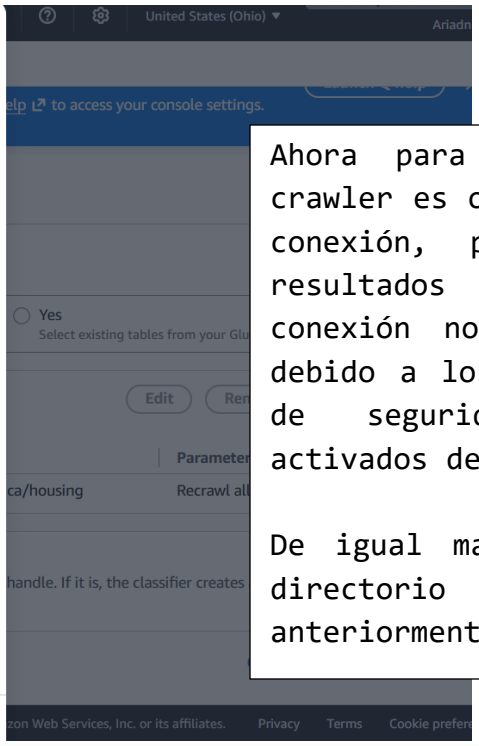
Description - *optional*

Enter a description

Descriptions can be up to 2048 characters long.

Para que se conecte la información de S3 a glue lo que se necesita ahora es crear un crawler





Ahora para la creación del crawler es opcional ponerle la conexión, pero me ha dado resultados en la que la conexión no pierde en foque debido a los múltiples grupos de seguridad y permisos activados de sus políticas.

De igual manera se ubica el directorio de S3 realizado anteriormente

Configure security settings

IAM role Q help

IAM permissions

To access other AWS services and resources you need an IAM role. Choose a role with the right permissions.

(•) Create default role ( ) Use another role

A new role will be created with the necessary permissions.

Role name

AWSGlueServiceRole-pract-housing

Maximum 64 characters. Use alphanumeric and +=,.@-_ characters.

Ahora se configura el nombre de su rol para que este mismo sea una conexión como la mencionada previamente

Lake Formation configuration - *optional*

Allow the crawler to use Lake Formation credentials for crawling the data source. Learn more.

[ ] Use Lake Formation credentials for crawling S3 data source

Checking this box will allow the crawler to use Lake Formation credentials for crawling the data source. If the data source is registered in another account, you must provide the registered account ID. Otherwise, the crawler will crawl only those data sources associated to the account. Only applicable to S3, Glue Catalog, Iceberg, and Hudi data sources.

ARADNA NÚÑEZ TORRES 16 de octubre de 2026

Set output and scheduling

Output configuration Q help

Target database

housing-practica

Clear selection | Add database

Table name prefix - *optional*

Type a prefix added to table names

Maximum table threshold - *optional*

This field sets the maximum number of tables the crawler is allowed to generate. In the event that this number is surpassed, the crawl will fail with an error. If not set, the crawler will automatically generate the number of tables depending on the data schema.

Type a number greater than 0

▶ Advanced options

Aquí se señala a donde se dirigida esta información especialmente a la tabla creada dentro de esta base de datos

Crawlers

A crawler connects to a data store, progresses through a prioritized list of classifiers to determine the schema for your data, and then creates metadata tables in your data catalog.

Crawlers (6) Q help

Last updated (UTC) October 16, 2026 at 19:12:44 | Action | Run | Create crawler

View and manage all available crawlers.

Filter crawlers 1

Name	State	Schedule	Last run	Last run times...	Log	Table changes fr...
crawler-climasp	Ready		Succeeded	October 5, 2026 a...	View log	-
crawler-test	Ready		Succeeded	October 7, 2026 a...	View log	1 created
kc-housing-tab	Ready		Succeeded	October 15, 2026 ...	View log	-
prac-housing-cra...	Ready		Succeeded	October 16, 2026 ...	View log	1 updated
supermarket-cral...	Ready		Succeeded	October 15, 2026 ...	View log	-
wine-crawler	Ready		Succeeded	October 15, 2026 ...	View log	-

Al correr el crawler se esperan unos minutos o segundos para que este comience a reconocer el entorno y los múltiples enlaces que se generaron para la tabla y el recibimiento de la información desde S3

Consulta 1

```
1 SELECT * FROM "AwsDataCatalog"."housing-practica"."housing" limit 10;
```

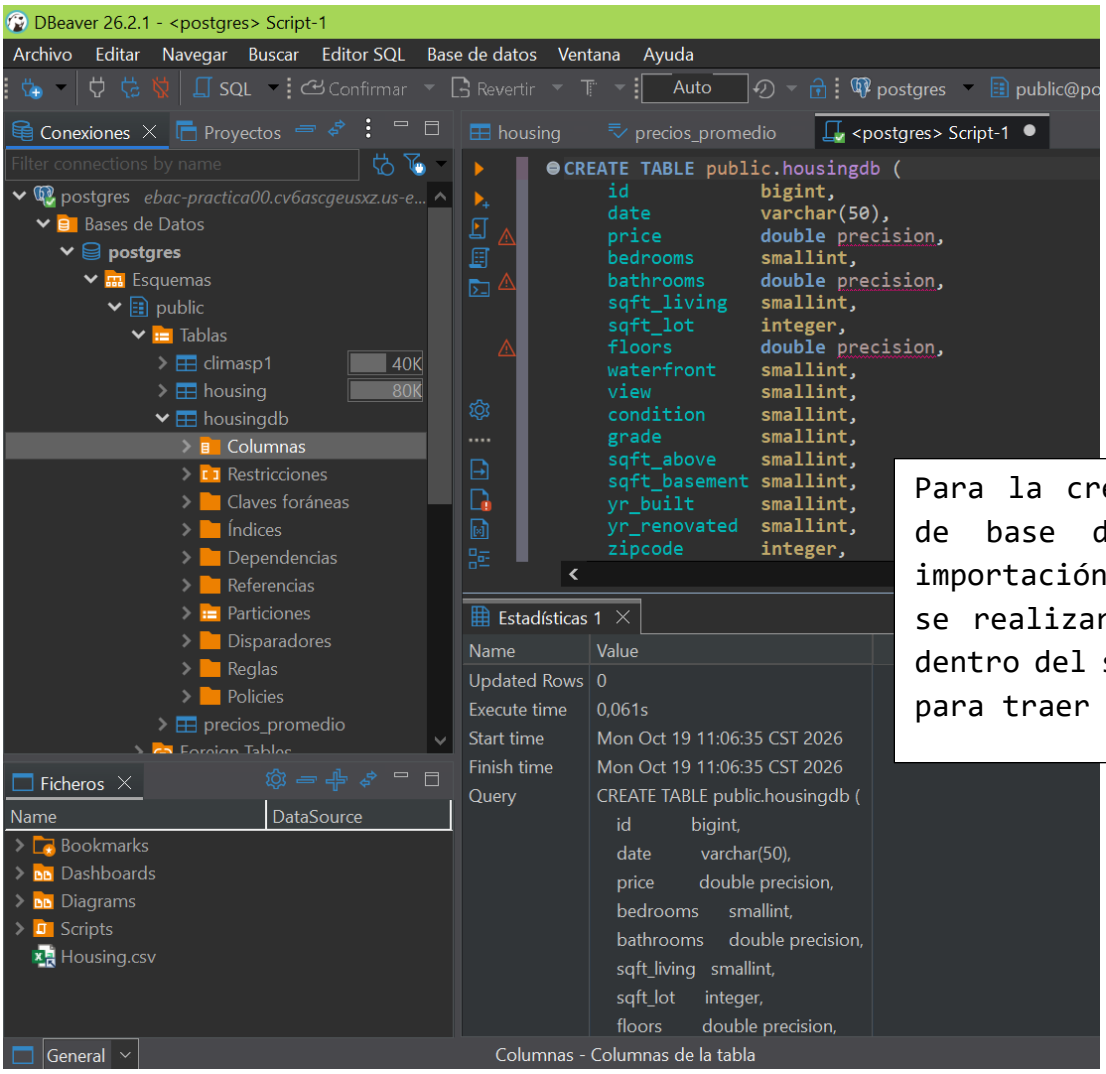
Resultados (10) Copiar | Descargar resultados en formato CSV

Filas de búsqueda 1

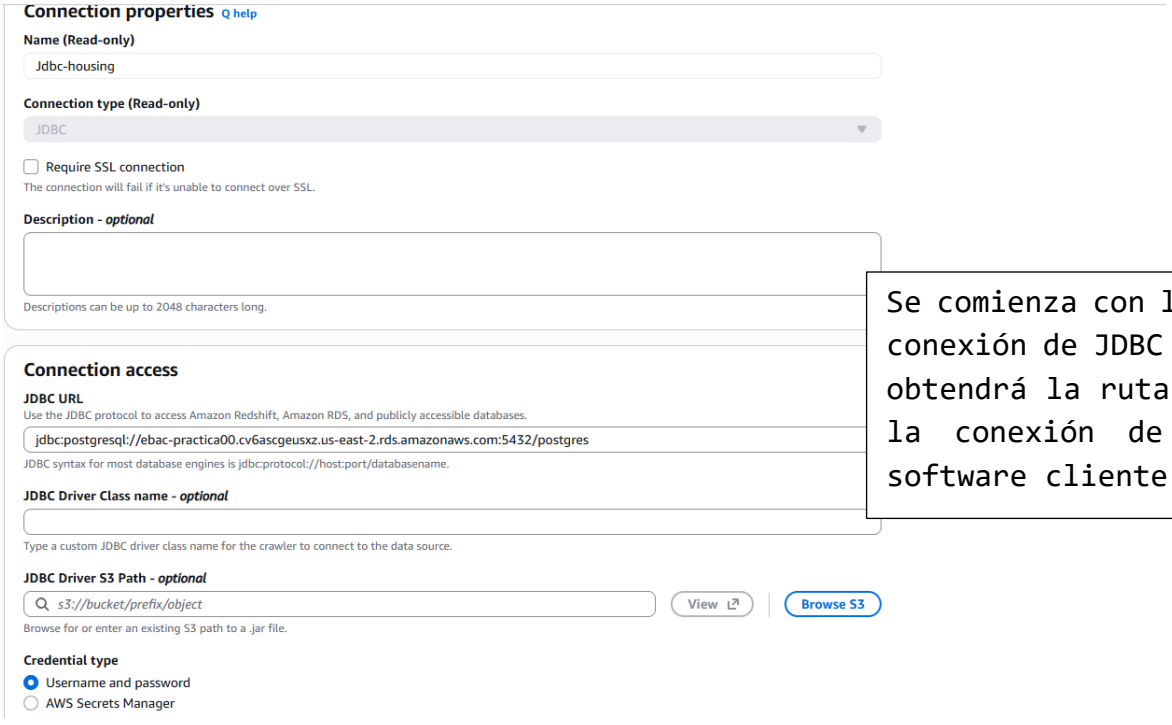
#	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterf
1	7129300520	20141013T000000	221900	3	1.0	1180	5650	1.0	0
2	6414100192	20141209T000000	538000	3	2.25	2570	7242	2.0	0
3	5631500400	20150225T000000	180000	2	1.0	770	10000	1.0	0
4	2487200875	20141209T000000	604000	4	3.0	1960	5000	1.0	0
5	1954400510	20150218T000000	510000	3	2.0	1680	8080	1.0	0
6	7237550310	20140512T000000	1.225e+006	4	4.5	5420	101930	1.0	0
7	1321400060	20140627T000000	257500	3	2.25	1715	6819	2.0	0
8	2008000270	20150115T000000	291850	3	1.5	1060	9711	1.0	0
9	2414600126	20150415T000000	229500	3	1.0	1780	7470	1.0	0
10	3793500160	20150312T000000	323000	3	2.5	1890	6560	2.0	0

Ya que el contenido se pasa a visualizar lo obtenido en ***Athena,*** así como la realización de una consulta básica

ARADNA NÚÑEZ TORRES 16 de octubre de 2026



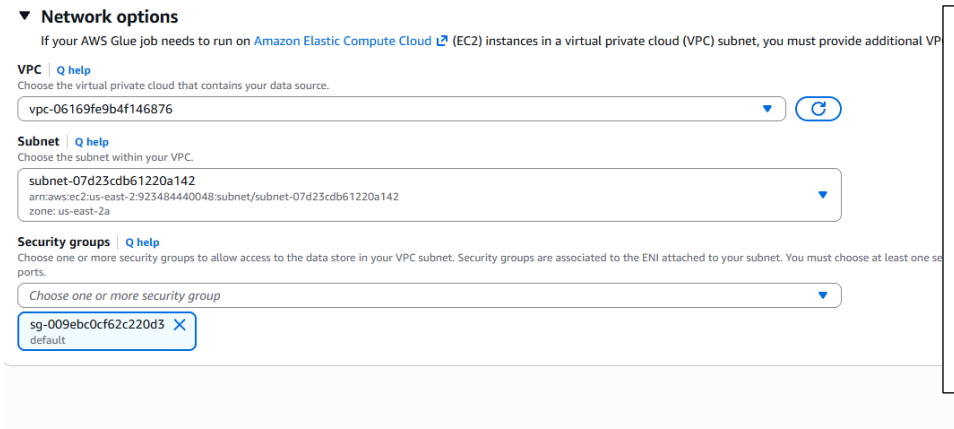
Para la creación de conexión de base de datos para la importación del archivo lo que se realizará será el esquema dentro del software de cliente para traer la información



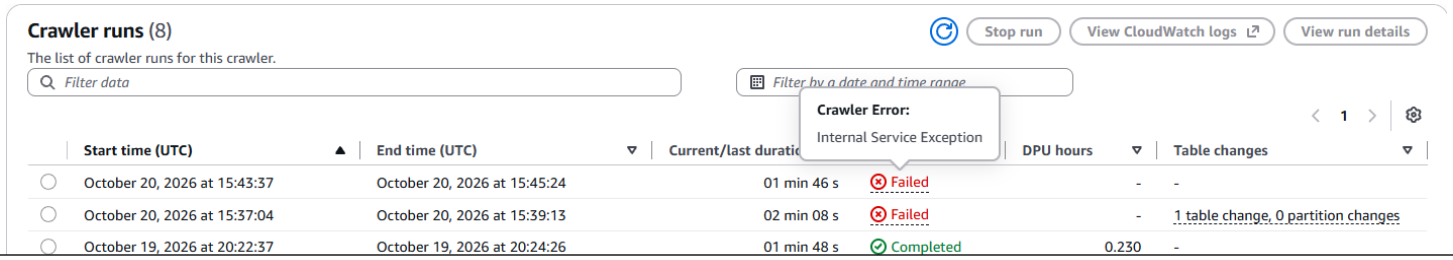
Se comienza con la creación de conexión de JDBC que es la que obtendrá la ruta que mantiene la conexión de RDS con el software cliente local

ARADNA NÚÑEZ TORRES

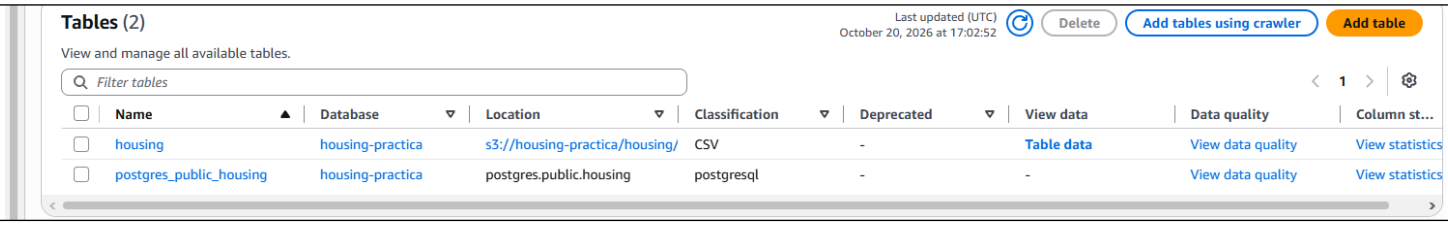
16 de octubre de 2026



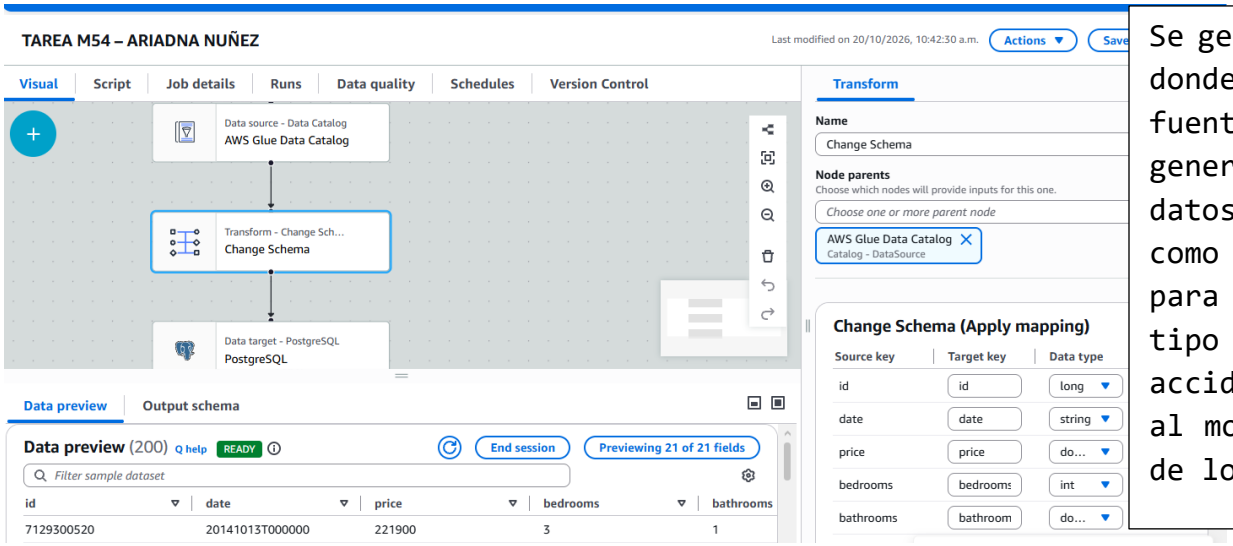
Como siguiente se utilizaran; la VPC que sera la que permita la conexión de la nube virtual hacia el entorno de Glue, ademas de señalar subredes y los grupos de seguridad previamente creados en ejercicios anteriores para fines de conexiones locales



Se crea un ***Crawler*** donde contenga dos fuentes de información, la primera desde ***S3*** que es donde tomara la información base para poder enviarla hacia el software seleccionado y la siguiente de ***JDBC*** que es la que dará la ruta de envío, es decir con el ***CRAWLER*** dará un envío de esto mas fluido. A pesar del error mostrado en su ***run*** es debido a problemas de AWS



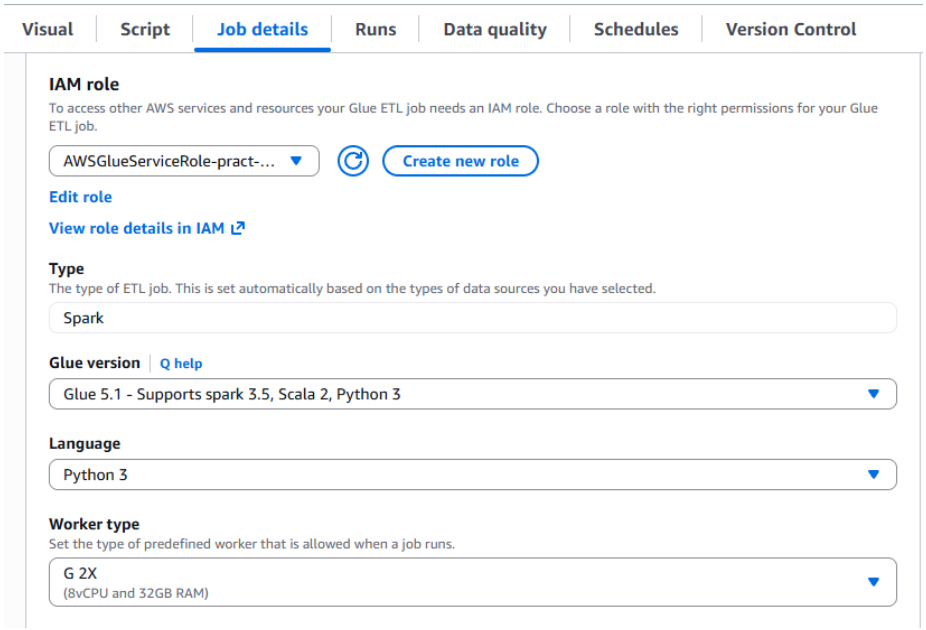
Como siguiente a checar es el catálogo de tablas de la base de datos donde por base al ***Crawler*** se creo la tabla que es la que será enviada a través de ETL al programa local



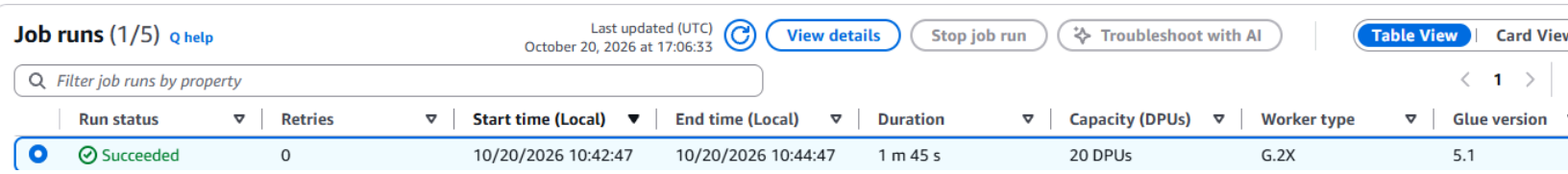
Se genera el job en el ETL donde se utilizara como fuente de datos la tabla generada en la base de datos de Glue que tiene como referencia desde S3, para en ello cambiar el tipo de dato que accidentalmente se cambió al momento de la lectura de los datos

ARADNA NÚÑEZ TORRES 16 de octubre de 2026

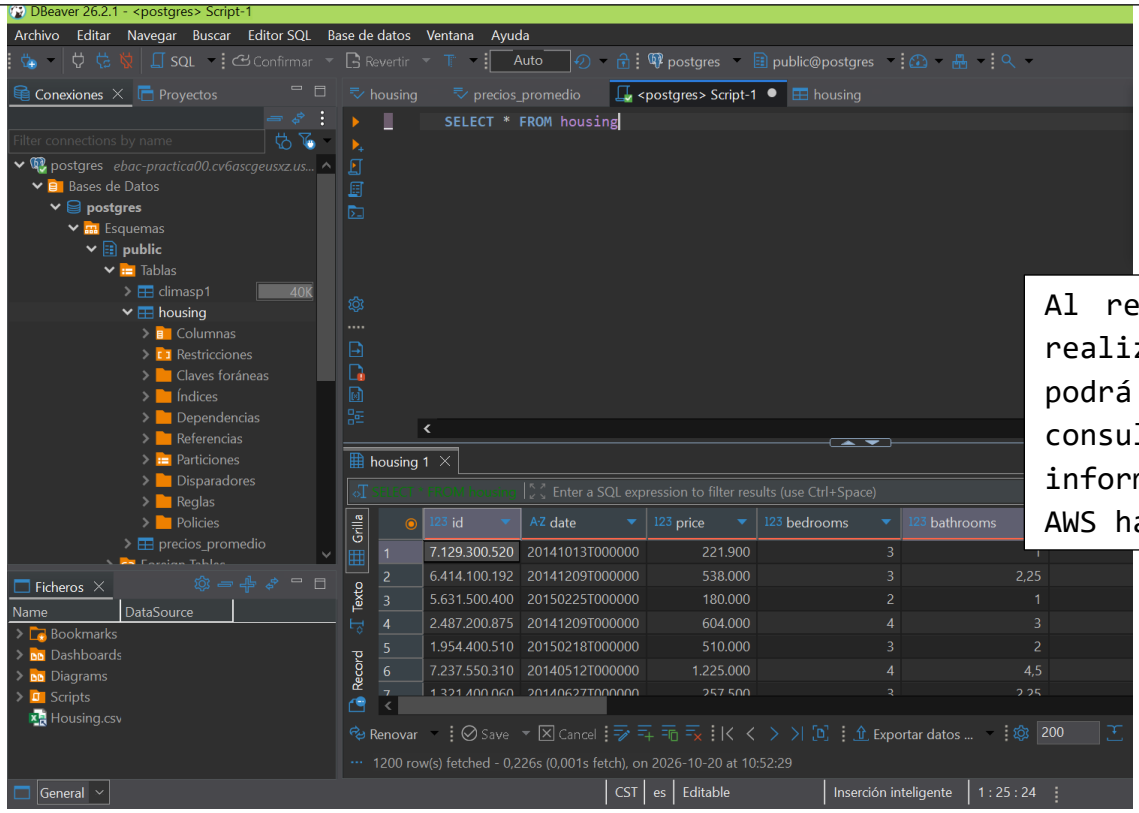
TAREA M54 – ARIADNA NUÑEZ



Antes de correr el ***Job*** lo que se realizara será seleccionar el rol de glue que es el generado en el crawler, ya que sigue la misma ruta, como siguiente será utilizar el G 2x que permitirá que el worker en el ***job*** haga su ***run***



Se guarda el ***job*** y corre el run con sus debidas modificaciones y señalizaciones de informacion ademas de su tipo de dato corregido en la parte de Visual ETL



Al refrescar la conexión realizada en DBeaver se podrá obtener con una consulta SQL la información enviada desde AWS hacia local

ARADNA NÚÑEZ TORRES 16 de octubre de 2026

EXPLORACION DE INFORMACIÓN IMPORTADA SOBRE SIMULACION DE BIENES RAICES

Se realizará la exploracion de informacion en base al archivo csv usado en AWS obteniendo su particion, reparticion y coalesce

```python
# Se importan secciones de pyarrow que apoyen a la extracci
from getpass import getpass
import adbc_driver_postgresql.dbapi as pg

password = getpass("Contraseña de RDS: OJKMTSHINee5")
uri = f"postgresql://postgres:{password}@ebac-practica00.cv

with pg.connect(uri) as conn, conn.cursor() as cur:
    cur.execute("SELECT * FROM public.housing")
    tabla = cur.fetch_arrow_table()

housingdf = tabla.to_pandas()
housingdf.info()
```

6] Python

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 172904 entries, 0 to 172903
Data columns (total 21 columns):
 #   Column         Non-Null Count   Dtype
---  ------         --------------   -----
 0   id             172904 non-null  int64
 1   date           172904 non-null  object
 2   price          172904 non-null  float64
 3   bedrooms       172904 non-null  int16
 4   bathrooms      172904 non-null  float64
```

Este es un método aprendido al uso de Pyarrow para el reconocimiento de la conexión que tiene PostgreSQL
Empezando por la importación de ***getpass*** que es el que permite obtener y solicitar las contraseñas de, en este caso ***RDS.***
Como siguiente es traer el driver / uri, que permite reconocer la conexión de Postgres para el uso de la base de datos desde un lugar virtual como la nube o un servidor local como ***dbeaver***

Ahora también pyarrow se utilizó para la transformación esquemática que se realizo en ***with*** con la variable *tabla* a un ***DataFrame*** de ***pandas*** que permite en futuros pasos visualizar esta etapa, por el momento se solicitó tener la información de los datos obtenidos para la ubicación del tipo de dato que maneja dicho archivo

EDA (Exploration Data Analysis)

```
#Importacion de librerias
import pandas as pd
#Importacion de archivo a utilizar
housing = pd.read_csv('databses/kc_house_data.csv')
#Chequeo del archivo
housing
```

✓ 0.0s

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sq
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	1050	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	1680	

Al pasar a esta parte lo que se realizará será principalmente un Análisis exploratorio de datos, permitiendo hacer su debida exploración solicitada al archivo otorgado para obtener respuestas sobre las preguntas generadas de la simulación de bienes raíces

```
#Primeros 5 resultados
housing.head()
```

✓ 0.0s

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0

5 rows × 21 columns

```
#Ultimos 5 resultados
housing.tail()
```

✓ 0.0s

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront
21608	263000018	20140521T000000	360000.0	3	2.50	1530	1131	3.0	0
21609	6600060120	20150223T000000	400000.0	4	2.50	2310	5813	2.0	0
21610	1523300141	20140623T000000	402101.0	2	0.75	1020	1350	2.0	0
21611	291310100	20150116T000000	400000.0	3	2.50	1600	2388	2.0	0

```
#Cinco muestras aleatorias del archivo presente
housing.sample(5)
```

✓ 0.0s

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	
1095	7202290160	20150114T000000	435000.0	3	2.50	1560	3987	2.0	0
8402	2616700450	20141107T000000	248000.0	3	1.75	1330	9831	1.0	0
14932	5608010050	20141229T000000	920000.0	4	2.50	3550	10233	2.0	0
4773	8929000170	20140616T000000	357186.0	2	1.75	1210	1040	2.0	0
15510	7686203620	20140925T000000	260000.0	4	2.50	2050	12500	1.0	0

Se comienza con el chequeo básico de obtención de 5 principales y últimos resultados del DataFrame generado a partir de Pandas. Así como lo es con la generación de 5 muestras aleatorias como se muestra a continuación permitiendo tener una idea sobre el archivo a laborar, a pesar que se lleva laborando desde AWS

Como siguiente se hace un chequeo sobre la información de cada columna que presenta, siendo estos los que se colocaron en AWS al momento de modificar el tipo de dato, debido a los errores que presento en ***Visual ETL*** al momento de enviar la información hacia el tarjet de ***PostgresSQL***

```
#Obtencion de informacion sobre el archivo presente
housing.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   id             21613 non-null  int64
 1   date           21613 non-null  object
 2   price          21613 non-null  float64
 3   bedrooms       21613 non-null  int64
 4   bathrooms      21613 non-null  float64
 5   sqft_living    21613 non-null  int64
 6   sqft_lot       21613 non-null  int64
 7   floors         21613 non-null  float64
 8   waterfront     21613 non-null  int64
 9   view           21613 non-null  int64
 10  condition      21613 non-null  int64
 11  grade          21613 non-null  int64
 12  sqft_above     21613 non-null  int64
 13  sqft_basement  21613 non-null  int64
```

```
#Obtencion de la estadistica básica
housing.describe().round(2)
```

✓ 0.0s

	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above
count	2.161300e+04	21613.00	21613.00	21613.00	21613.00	21613.00	21613.00	21613.00	21613.00	21613.00	21613.00	21613.00
mean	4.580302e+09	540088.14	3.37	2.11	2079.90	15106.97	1.49	0.01	0.23	3.41	7.66	1788.39
std	2.876566e+09	367127.20	0.93	0.77	918.44	41420.51	0.54	0.09	0.77	0.65	1.18	828.09
min	1.000102e+06	75000.00	0.00	0.00	290.00	520.00	1.00	0.00	0.00	1.00	1.00	290.00

Por último por parte del EDA lo que se obtendrá es la información sobre las estadísticas básicas donde indicaran desde, su conteo de datos, su promedio, desviación estándar, su mínimo y máximo

Análisis adicionales + preguntas dirigidas al negocio

Preguntas generadas dirigidas al negocio

- ¿Cuales son las casas que mejor condicion tienen?
- ¿De que dependen el año de construccion con la calidad / grado?
- ¿Cuales son los precios promedio de casas con 3 habitaciones

Análisis Bivariado

```
#Se examina una de las variables a identificar los resultados
housingdf.groupby(['zipcode','grade','condition'])['price'].mean().head(20)
```

✓ 0.0s

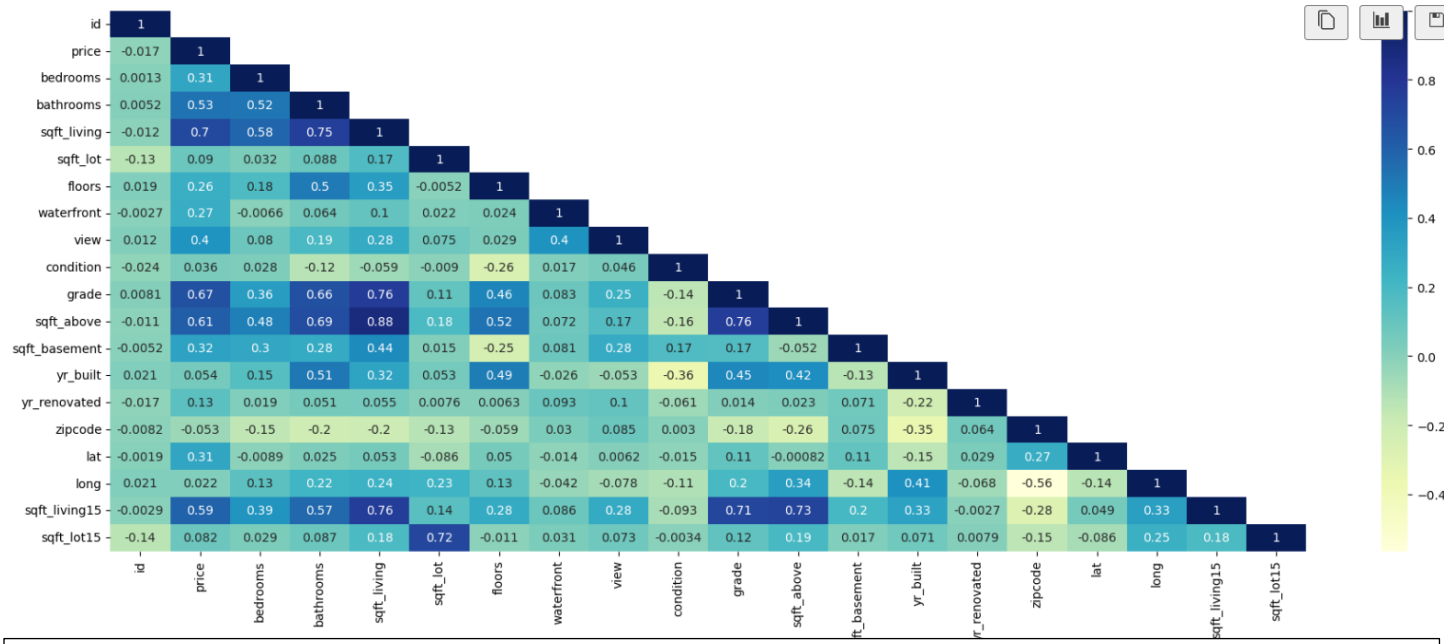
zipcode	grade	condition	
98001	5	3	157475.000000
		4	189458.333333
	6	3	197941.538462
		4	217425.307692
		5	198700.000000
	7	2	232587.500000
		3	241650.514925
		4	245960.921875
		5	248335.000000
	8	3	335814.322581
		4	349342.307692
		5	360000.000000
	9	2	850000.000000
		3	445166.310345
		4	270000.000000
	10	3	543380.000000
98002	5	3	157375.000000
		4	137960.000000
		5	181679.000000
	6	3	177492.857143

En este análisis se utilizaron ciertas variables donde permite entender por que estas casas fueron de las mas solicitadas para visitar. Varios de los visitantes consideraron que tienen una buena calidad, sin embargo, la condición de algunas es baja debido a las pocas restauraciones que se realizaron

Análisis correlacional

```
#Se grafican los resultados obtenidos
plt.figure(figsize= (22,8))
corr_house = corr = housingdf.corr(method= 'pearson',numeric_only= True)
house_lt = corr_house.where(np.tril(np.ones(corr_house.shape)).astype(bool))
hmap = sns.heatmap(house_lt, cmap = 'YlGnBu', annot = True)
```

✓ 0.6s



Este análisis tiene un propósito que mas para el cliente es para comprender por que ciertas casas tienen a tener ciertas características que las hace mas caras o mas económicas, muchas veces es por los m2 o ft, también por sus pisos, habitaciones, baños, de igual manera si contiene sótano, ático o alguna vista hacia algún campo o vista acuática. Distintos factores que se le pueden mencionar al cliente para que se le dé a entender que igualmente la casa / lugar es apto para una reconstrucción o demás

Análisis agrupado

```
#Agrupacion de datos para obtencion de la media
print('Precio promedio de una casa con 3 habitaciones')
housingdf[housingdf.bedrooms == 3]['price'].mean()
```

✓ 0.0s

```
Precio promedio de una casa con 3 habitaciones

np.float64(466232.07848127035)
```

Por otro tipo de vivienda que se pregunto fue por las que tienen tres habitaciones, que en su mayoría los precios rondan entre el que esta señalado

ARADNA NÚÑEZ TORRES 16 de octubre de 2026

KPI´s

Datos que permiten al cliente entender el negocio

Se realizaron varios apartados individuales que permita ser categóricos aproximados generales de varios datos para que se dé a entender a qué grupos de personas está dirigida esta información

```
#Obtencion de tamaño promedio de casa
print('Tamaño promedio de casa basada para una a dos personas')
housingdf['living_m2'] = housingdf['sqft_living'] * 0.093
housingdf['lot_m2'] = housingdf['sqft_lot'] * 0.093
housingdf['m2'] = housingdf['living_m2'] * housingdf['lot_m2']

housingdf.groupby(['zipcode','m2','price','bathrooms','floors'])['bedrooms'].min().sort_values(ascending = True).head(30)
```

✓ 0.0s

```
Tamaño promedio de casa basada para una a dos personas
```

zipcode	m2	price	bathrooms	floors	
98001	3.116266e+04	139950.0	0.00	1.0	0
98024	5.235888e+04	142000.0	0.00	1.0	0
98102	1.262486e+05	1095000.0	0.00	3.5	0
98031	1.712549e+05	355000.0	0.00	2.0	0
98133	1.244794e+04	380000.0	0.00	3.0	0

```
print('Tamaño promedio de casa basada para una familia mediana (aprox. 4 a 6 personas)')
housingdf.groupby(['zipcode','m2','price','bedrooms','floors'])['bathrooms'].mean().sort_values(ascending = False).head(10)
```

✓ 0.0s

```
Tamaño promedio de casa basada para una familia mediana (aprox. 4 a 6 personas)
```

zipcode	m2	price	bedrooms	floors	
98053	3.604006e+07	2280000.0	7	3.0	8.00
98102	2.876484e+06	7700000.0	6	2.5	8.00
98039	2.683688e+06	6885000.0	6	2.0	7.75
98144	2.278250e+05	450000.0	9	2.0	7.50
98166	2.695432e+06	800000.0	7	2.0	6.75
98040	1.089562e+06	4668000.0	5	1.0	6.75
98075	5.931337e+05	1180000.0	6	2.0	6.50
98027	8.175239e+06	2238890.0	5	2.0	6.50
98006	1.507856e+06	3300000.0	5	2.0	6.25
98177	4.801632e+06	2888000.0	5	2.0	6.25

```
print('Tamaño promedio de casa basada para familias grandes (aprox. 10 a 12 personas)')
housingdf.groupby(['zipcode','m2','price','bathrooms','floors'])['bedrooms'].max().sort_values(ascending = False).head(10)
```

✓ 0.0s

```
Tamaño promedio de casa basada para familias grandes (aprox. 10 a 12 personas)
```

zipcode	m2	price	bathrooms	floors	
98103	84068.28000	640000.0	1.75	1.0	33
98106	128697.12000	520000.0	3.00	2.0	11
98105	94580.27460	660000.0	3.00	2.0	10
98004	433512.09720	1148000.0	5.25	1.0	10
98006	371989.51146	650000.0	2.00	2.0	10
98105	157844.25000	1280000.0	4.50	2.0	9
	109268.00640	934000.0	3.00	2.0	9
	220090.75704	1400000.0	4.00	2.5	9
98144	227825.03880	450000.0	7.50	2.0	9
98102	140044.60800	700000.0	3.00	2.0	9

Las agrupaciones en su mayoría se basaron en las habitaciones, ya que de esa forma puede traer la información de viviendas que cuentan con bastantes y con las que solo son de negocio o donde solo residirían pocas personas

```
#Obtencion de precios promedio de casas
print('Precios promedio')
housingdf.groupby(['price','bedrooms','bathrooms','floors','yr_built','m2'])['grade'].mean().sort_values(ascending = False).head(10)
```

✓ 0.0s

```
Precios promedio
```

price	bedrooms	bathrooms	floors	yr_built	m2	
3200000.0	6	5.00	2.5	1933	1.117624e+06	13.0
6885000.0	6	7.75	2.0	2001	2.683688e+06	13.0
2479000.0	5	3.75	2.5	1922	4.417477e+05	13.0
2340000.0	4	2.50	1.0	1988	1.370002e+06	13.0
2888000.0	5	6.25	2.0	1965	4.801632e+06	13.0
2385000.0	4	4.00	2.0	2000	7.279317e+05	13.0
2415000.0	5	4.75	2.0	1996	1.652737e+06	13.0
2983000.0	5	5.50	2.0	2001	1.209521e+06	13.0
3800000.0	5	5.50	1.0	1978	2.612188e+06	13.0
7700000.0	6	8.00	2.5	1910	2.876484e+06	13.0

Añadiendo como dato extra y de conocimiento desde cuando se construyó que es lo que en su mayoría de ocasiones se da a conocer debido a la apariencia o los materiales que presenta el lugar y vivienda